



Arm® Firmware Framework v1.2 for Arm® A-profile Architecture Compliance Suite

Version v1.2

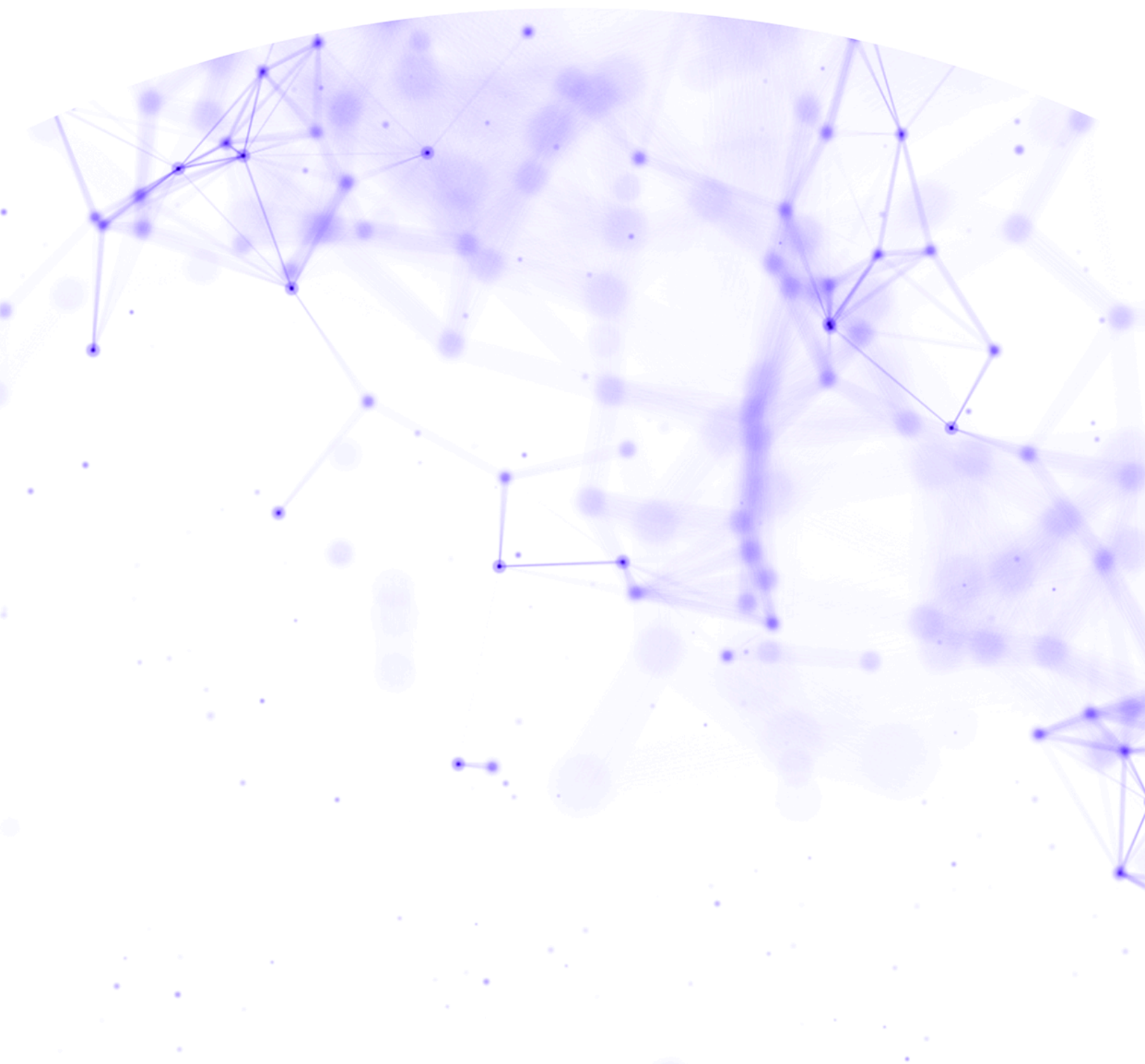
Validation Methodology

Non-Confidential

Copyright © 2021–2022, 2024, 2026 Arm Limited (or its affiliates).
All rights reserved.

Issue 02

102411_0102_02_en



Arm® Firmware Framework v1.2 for Arm® A-profile Architecture Compliance Suite Validation Methodology

This document is Non-Confidential.

Copyright © 2021–2022, 2024, 2026 Arm Limited (or its affiliates). All rights reserved.

This document is protected by copyright and other intellectual property rights.

Arm only permits use of this document if you have reviewed and accepted [Arm's Proprietary Notice](#) found at the end of this document.

This document (102411_0102_02_en) was issued on 2026-07-20. There might be a later issue at <https://developer.arm.com/documentation/102411>

The product version is v1.2.

See also: [Proprietary Notice](#) | [Product and document information](#) | [Useful resources](#)

Start reading

If you prefer, you can skip to [the start of the content](#).

Intended audience

This book is written for engineers who are specifying, designing, or verifying an implementation of the Arm® Firmware Framework for Arm®v8–A architecture.

Inclusive language commitment

Arm values inclusive communities. Arm recognizes that we and our industry have used language that can be offensive. Arm strives to lead the industry and create change.

We believe that this document contains no offensive language. To report offensive language in this document, email terms@arm.com.

Feedback

Arm welcomes feedback on this product and its documentation. To provide feedback on the product, create a ticket on <https://support.developer.arm.com>.

To provide feedback on the document, fill the following survey: <https://developer.arm.com/documentation-feedback-survey>.

Contents

1. Overview to ACS.....	4
1.1 Abbreviations.....	4
1.2 Arm Firmware Framework for Arm A-profile.....	4
1.3 Architecture Compliance Suite.....	5
1.4 Test suite components.....	6
1.5 Directory structure.....	7
1.6 Feedback, contributions, and support.....	8
2. Validation methodology.....	9
2.1 Test layering details.....	9
2.2 Integrating the test suite with the SUT.....	11
2.3 Test execution flow.....	11
2.4 Test configuration.....	15
2.5 Test endpoint boot.....	15
2.6 MP execution context setup.....	16
2.7 Error handling.....	17
2.8 Running ACS with Linux OS.....	17
2.9 Analyzing test results.....	18
Proprietary Notice.....	20
Product and document information.....	22
Product status.....	22
Revision history.....	22
Conventions.....	23
Useful resources.....	26

1. Overview to ACS

This chapter introduces the features and components of Architecture Compliance Suite for Arm® Firmware Framework for Arm® A-profile.

1.1 Abbreviations

This section lists the abbreviations that are used in this document.

Table 1-1: Abbreviations and expansions

Abbreviation	Expansion
ABI	Application Binary Interface
ACS	Architecture Compliance Suite
API	Application Programming Interface
BSS	Block Started by Symbol
CPU	Central Processing Unit
EL	Exception Level
EP	Endpoint
FF-A	Arm Firmware Framework for A-profile
GOT	Global Offset Table
MP	Multi-Processor
OSPM	Operating System Power Management
PAL	Platform Abstraction Layer
PM	Partition Manager (represents both SPM and hypervisor)
PSCI	Power State Coordination Interface
PVM	Primary Virtual Machine (implementing FF-A primary scheduler functionality)
SP	Secure Partition
SPM	Secure Partition Manager
SPMC	SPM Core
SPMD	SPM Dispatcher
SUT	System Under Test
SVM	Secondary Virtual Machine
UP	Uni-Processor
VAL	Validation Abstraction Layer
VM	Virtual Machine
TPM CRB	Trusted Platform Module Command Response Buffer

1.2 Arm Firmware Framework for Arm A-profile

Arm Firmware Framework for A-profile (Arm FF-A) describes a software architecture that achieves the following goals:

- Applies the Virtualization Extension to isolate software images provided by different vendors.
- Describes the interfaces that standardize communication between the various software images. This includes communication between images in the Secure world and Normal world.

Arm FF-A also goes beyond the mentioned goals to ensure that the interfaces are used to standardize communication:

- In the absence of Virtualization Extensions in the Secure world, this aspect provides a migration path for existing Secure world software images to a system that implements the Virtualization Extension in the Secure state.
- Between Virtual Machines (VMs) managed by a hypervisor in the Normal world, the Virtualization Extensions in the Secure state mirrors its counterpart in the Non-secure state. The hypervisor uses the FF-A interfaces to enable communication between the VMs that it manages.

The main components of Arm FF-A are:

- A Partition Manager (PM), which manages partitions is the hypervisor in Normal world and the Secure Partition Manager (SPM) in Secure world.
- One or more partitions that are sandboxes created by the PM could be VMs in Normal world or Secure world. The VMs in Secure world are called Secure Partitions (SP).
- Application Binary Interfaces (ABIs) that partitions can invoke to communicate with other partitions.
- A partition manifest describes system resources, requirements, implemented services, and attributes related to governing the runtime behavior of a partition.



In this document, the terms Endpoint (EP) and partition are used interchangeably.

For more information on Arm FF-A, see the [Arm Firmware Framework for Arm A-profile](#) specification.

1.3 Architecture Compliance Suite

Architecture Compliance Suite (ACS) contains a set of functional tests, demonstrating the invariant behaviors that are specified in the architecture specification. It is used to ensure architecture compliance of the implementations to Arm FF-A specification.

These ACS tests cover checks for the following categories of features, with each suite covering a different area of the architecture.

Table 1-2: Test suite categories and their descriptions

Suite name	Covered features	Description
setup_discovery	FF-A set up and discovery interfaces, status reporting interfaces, and partition initialization.	Directed test cases verifying the implementation of Arm FF-A for the FF-A set up and discovery interfaces, and endpoint set up requirements such as initialization of partition, setting up the Multi-Processor (MP) execution contexts for partition, and Uni-Processor (UP) migrate capability.
direct_messaging	FF-A direct messaging interfaces, status reporting interfaces, and FF-A Central Processing Unit (CPU) cycle management interfaces.	Directed test cases verifying the implementation of Arm FF-A for the FF-A direct messaging interfaces and FF-A CPU cycle management interfaces.
indirect_messaging	FF-A indirect messaging interfaces, status reporting interfaces, and FF-A CPU cycle management interfaces.	Directed test cases verifying the implementation of Arm FF-A for the FF-A indirect messaging interfaces and FF-A CPU cycle management interfaces.
memory_manage	FF-A memory management interfaces and status reporting interfaces.	Directed test cases verifying the implementation of Arm FF-A for the FF-A memory management interfaces.

Suite name	Covered features	Description
Notifications	FF-A notification interfaces.	Directed test cases verifying the implementation of Arm FF-A for the FF-A notification interfaces.
Interrupts	FF-A interrupt management.	Directed test cases verifying the implementation of Arm FF-A for the FF-A interrupt management.
tpm	TPM CRB over FF-A	Directed test cases verifying the implementation of TPM CRB over FF-A.



The test suite contains tests that have checks embedded within the test code. To view the list of tests and the different categories of features that are checked for compliance, see the `testcase_checklist.md` document in the `docs/` directory.

1.4 Test suite components

The following table describes the test suite components.

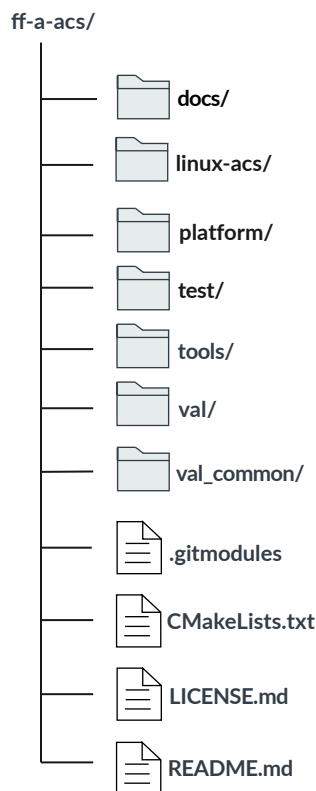
Table 1-3: Test suite components

Component	Description
Tests	Self-checking C tests.
Substructure	Test supporting layers consist of a framework and libraries set up as: • Tools to build the tests • Validation Abstraction Layer (VAL) library • Platform Abstraction Layer (PAL) library
Documentation	Consists of Porting Guide, Test scenario document, and Validation Methodology document.

1.5 Directory structure

The following figure shows the top-level directory of the test suite when the release package is downloaded from GitHub.

Figure 1-1: Top-level directory structure of FF-A test suite



The following table describes the components of the test suite directory.

Table 1-4: Test suite directory

Components	Description
ff-a-acs	Top-level test suite directory.
docs	Contains the test suite documentation.
linux-acs	Contains the Linux kernel driver files.
platform	Contains files to form the PAL. PAL is the closest to hardware and is aware of the underlying hardware details. Since this layer interacts with hardware, it must be ported or tailored to specific hardware required for system components present in a platform. For more information on porting setups see, <code>docs/porting_guide.md</code> document. This layer is also responsible for presenting a consistent interface to the VAL required for the tests.
test	Contains the Arm FF-A tests.
tools	Contains CMake files and scripts that are used to generate test binaries.
val	Contains subdirectories for the VAL libraries. This layer provides a uniform and consistent view of the available test infrastructure to the tests in the test suite. The VAL makes appropriate calls to the PAL to achieve this functionality. This layer is not required to be ported when the underlying hardware changes.

Components	Description
val_common	Contains the VAL library files.
.gitmodules	Contains the Git submodules.
CMakeLists.txt	Contains information on the CMake build support.
License	Contains the licensing information for the project.
README.md	Contains information on Arm FF-A test suite.

1.6 Feedback, contributions, and support

For feedback, use the GitHub Issue Tracker that is associated with this repository.

For support, send an email to support-ff-a-ac@arm.com with the details.

Arm licensees can contact Arm directly through their partner managers.

Arm also welcomes code contributions through GitHub pull requests. See, GitHub documentation on how to raise pull requests.

2. Validation methodology

This chapter describes the validation methodology for the Architecture Compliance Suite.

2.1 Test layering details

Arm FF-A ACS defines three ACS Secure endpoints in Secure world. It also defines one or three ACS Non-secure endpoints (Virtual Machines) in Normal world depending on the presence of Non-secure hypervisor in the system.

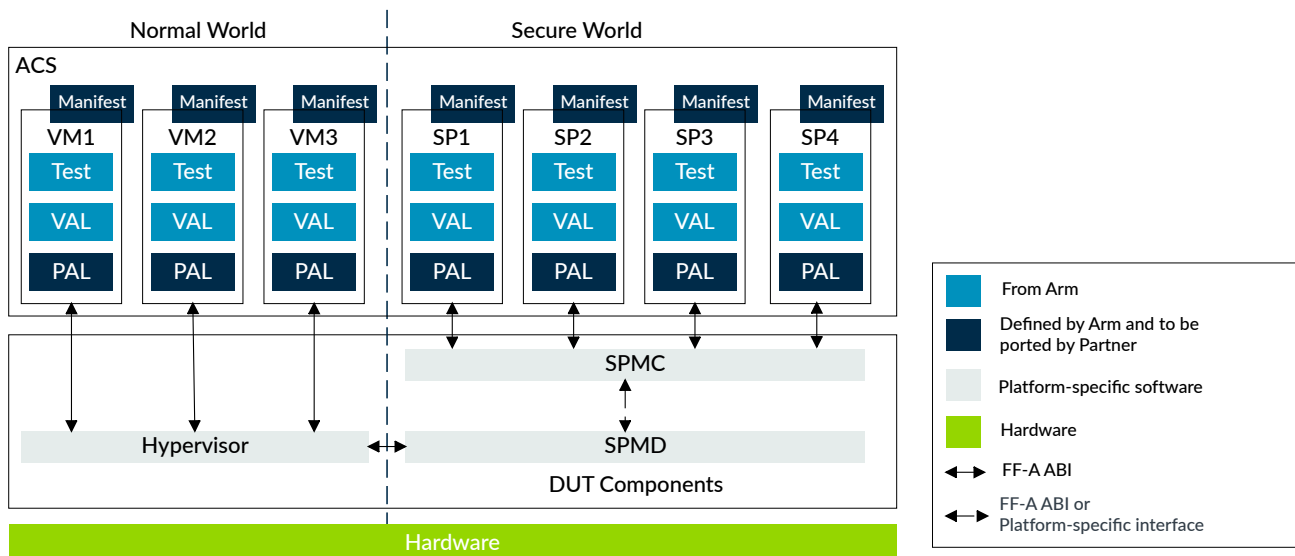
Each of these endpoints provide self-checking and portable C-based tests with directed stimulus. These tests use the layered software stack approach to enable porting across different test platforms.

The constituents of the layered stack are:

- Tests
- VAL
- PAL

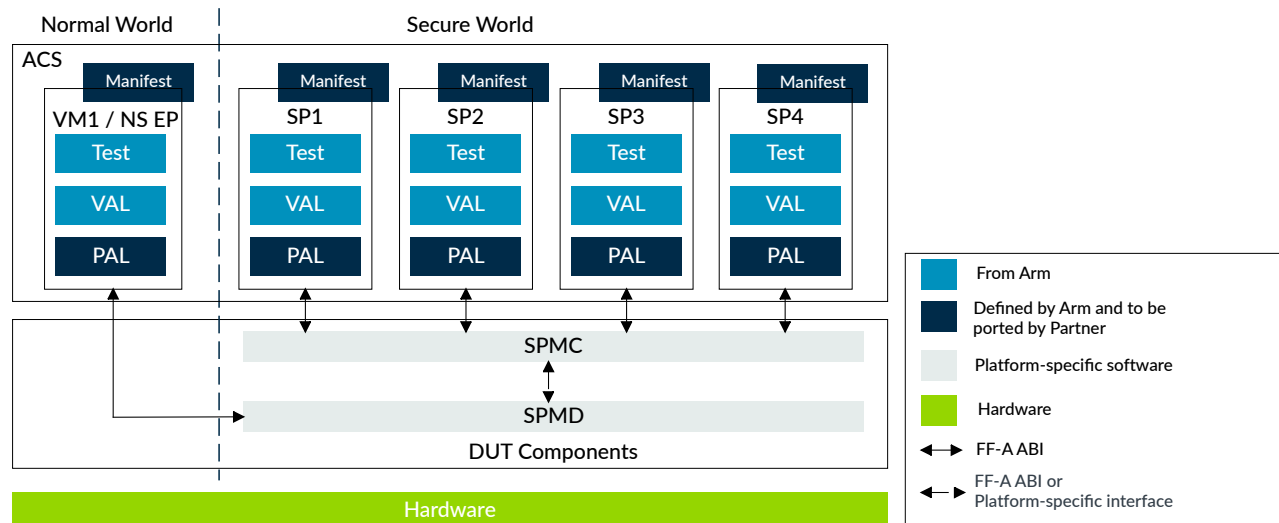
The following figure shows the System Under Test (SUT) when Non-secure hypervisor component is present in the system.

Figure 2-1: SUT when Non-secure hypervisor is present



The following figure shows the SUT when a Non-secure hypervisor component is absent in the system.

Figure 2-2: SUT when Non-secure hypervisor is absent



The following table describes the constituents of the layered stack.

Table 2-1: Layered software stack components

Layer	Description
Tests	A set of C and assembly-based directed tests verifies the implementation against a test scenario that is described by the Arm FF-A specification has Secure and Non-secure test checks. These tests are launched from Normal world test endpoint. Dedicated instance of client-server test runs on different test VM or SP, depending on the test scenario. These tests are abstracted from the underlying hardware platform by the VAL. This implies that porting a test for a specific target platform is not required. Each test endpoint is also provided with a manifest file describing the resources that it needs for to boot and function.
VAL	This layer provides a uniform and consistent view of the available test infrastructure to the tests in the test pool by making appropriate calls to the PAL. It is designed in a way that it can be used from both Secure and Non-secure sides. This layer does not require porting when the underlying hardware changes.
PAL	This layer is the closest to the hardware and is aware of the platform details. It is responsible for presenting the hardware through a consistent interface to VAL. This layer must be ported to the specific hardware present in the platform. The PAL is designed in a way that it can be used from both Secure and Normal worlds.

2.2 Integrating the test suite with the SUT

The test compilation flow creates the following libraries that you must integrate with the SUT software.

Table 2-2: Integrating test binaries

Test binaries	Integration and loading
Secure world test endpoint binaries: • <BUILD_DIR>/output/sp1.bin • <BUILD_DIR>/output/sp2.bin • <BUILD_DIR>/output/sp3.bin • <BUILD_DIR>/output/sp4.bin	These Secure test binaries must be integrated and loaded in Secure world using platform-specific SPM Core (SPMC) component with the information provided in their respective test endpoint manifest files available in <code>platform/manifest/<platformName>/</code> . These SPs can be executed at SELO or SEL1 depending on whether SPMC is implemented at SEL1 or SEL2.
Normal world test dispatcher endpoint (VM1) binary: • <BUILD_DIR>/output/vm1.bin	This binary can be integrated in the following ways depending on availability of Non-secure hypervisor: • If Non-secure hypervisor is present, it must integrate and load this binary as one of the VMs in Normal world. VM1 must be run at NS-EL1 in this configuration. • If Non-secure hypervisor is absent, Secure world components must integrate and load this binary as Normal world image (Linux OS kernel image). VM1 can be run at NS-EL1 or NS-EL2 in this configuration.
Normal world secondary test endpoint (VMs) binaries: • <BUILD_DIR>/output/vm2.bin • <BUILD_DIR>/output/vm3.bin	The Secondary Virtual Machine (SVM) binaries must be integrated and loaded in Normal world using the Non-secure hypervisor with the information provided in their respective test endpoint manifest files which is available in <code>platform/manifest/<platformName>/</code> . Secondary VMs must run at NS-EL1.



For more information on running ACS with Linux OS, see the [2.8 Running ACS with Linux OS](#) on page 17.

2.3 Test execution flow

This section provides the details of the test execution flow for Arm FF-A tests.

The sequence of operations for Arm FF-A test execution flow is as follows:

1. The SUT boots to an environment that enables the test functionality. This implies that the SPM and hypervisor are initialized, and ACS test endpoints (except test dispatcher VM) are ready to accept the requests.
2. SUT boot software gives control to the test dispatcher VM in Normal world. The dispatcher VM then invokes the `test_dispatcher` function.
3. Dispatcher VM launches the `<testname>_testentry` function for each test drives the overall test regression.
4. The dispatcher also makes VAL (and in turn PAL) calls to save and reports each of the test results. The dispatcher VM then sends the message to required test endpoints to release it for executing appropriate test functions as per the test needs.

Based on the test scenario, different test endpoints communicate with each other using FF-A ABIs that are defined in the specification and report the test results using VAL print API (in turn PAL API ported to the specific platform). Each test scenario is driven using dedicated client-server test functions and they are:

- Available in `<testname>_client.c` and are suffixed with `_client` label. Based on test needs, client functions are executed in any of the available test endpoints.

- Available in `<testname>_server.c` and are suffixed with `_server` label. The server functions are invoked on specified test endpoints by the client test function. `Test_entry` function and information about different requirements for endpoint interactions are specified in `<testname>_testentry.c` file of the test.
- After completion of the client-server test functions, dispatcher VM collects test status and prints it to console.

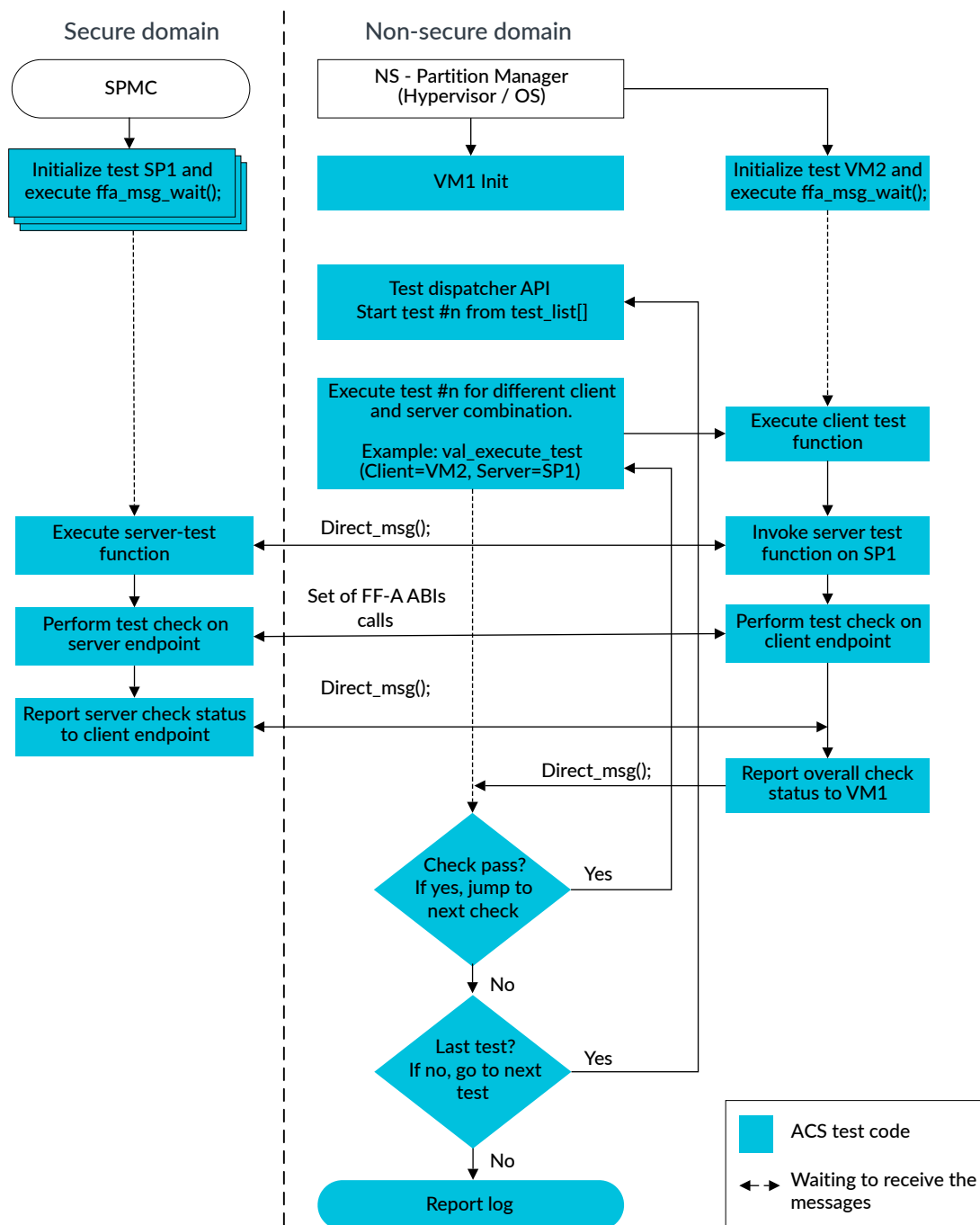
The tests query the VAL layer to get the necessary information to run the tests. This information can include memory maps, interrupt maps, and hardware controller maps.



To facilitate test reporting and management aspects, the Arm FF-A system contains UART for printing the status of tests. If a display console is not available, PAL can be updated to make the test results available to the external world through other means.

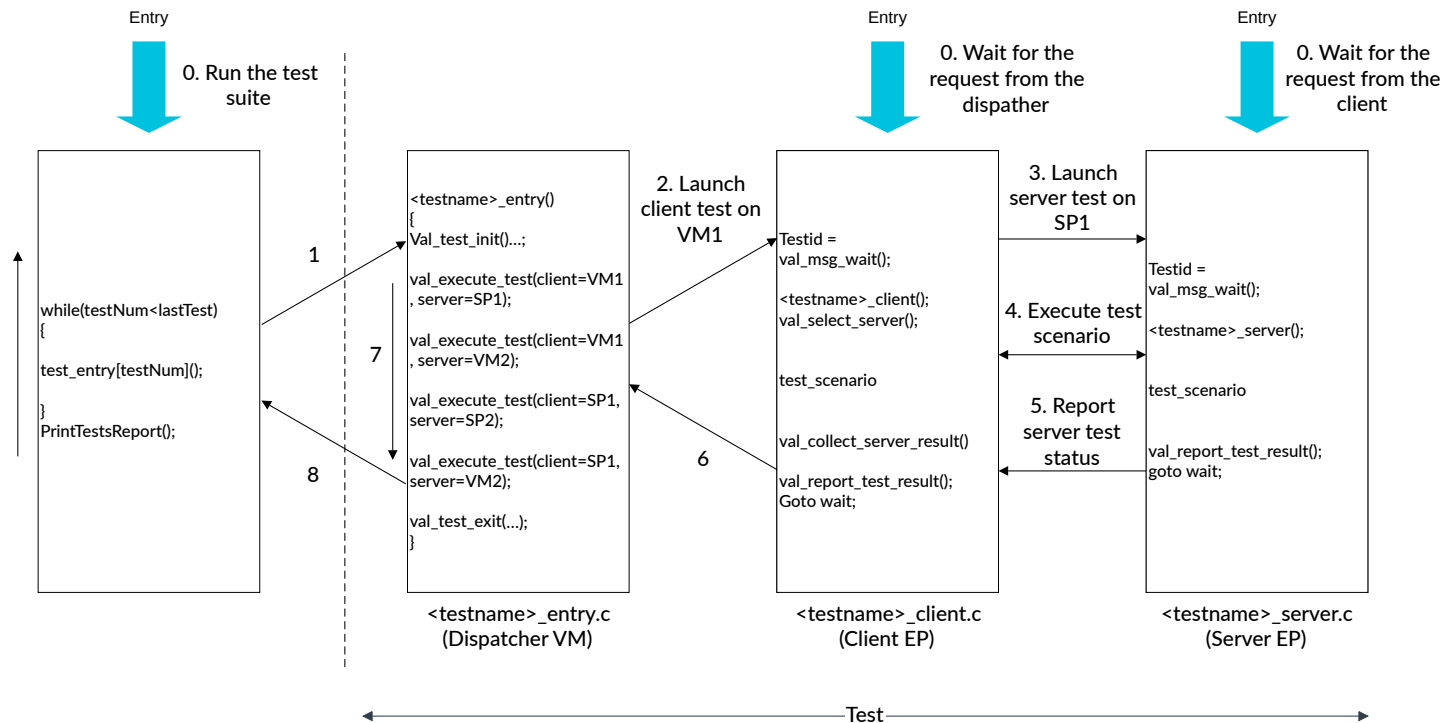
The following figure shows the test execution flow for the Secure and the Non-secure domains.

Figure 2-3: Test execution flow



The following figure is an example of a code snippet for test execution flow.

Figure 2-4: Example code snippet for execution flow



The execution steps for the code snippet are described as follows:

1. Dispatcher VM launches the `<testname>_entry` function for each test drives the overall test regression.
2. On test entry, dispatcher VM performs the test initialization as part of `val_test_init` function. Example of such initialization is printing the test name onto the console, enabling the watchdog timer. The dispatcher VM then selects the client and server pair combination that must be used for executing the test sequence as part of `val_execute_test` function. This function launches client test functions onto the client endpoint specified using logical ID. This function also passes the test-related metadata such as the current test client and server logical IDs and test number to the client endpoint.
3. Client launches server test function onto the server endpoint as part of the `val_select_server_fn` function.
4. Client-server endpoint executes the test sequence that is written for the given test scenario.
5. On completion of the test scenario sequence, client endpoint collects the server test function status.
6. Client reports the final test status for the chosen client-server pair.
7. Dispatcher VM repeats steps 3 to 6 for the next pair of client and server.
8. Dispatcher VM performs the test exit sequence as part of the `val_test_exit()` and prints the overall test status onto console.
9. Dispatcher VM repeats the preceding steps for all other tests and prints the overall test summary for the suite.



All the connections between test dispatcher VM to client EP and client EP to server EP happen using direct messaging.

2.4 Test configuration

Several Arm FF-A interfaces are common for both the Security state endpoints, the test scenarios for such interfaces must be repeated with different endpoint combinations.

For example, direct messaging test must be repeated for VM to SP, SP to SP, and VM to VM communication. The ACS uses the test configuration mechanism to help in the reuse of test code for multiple endpoint combinations as follows:

1. The test configuration is based on runtime selecting the client-server pair. The `<testname>testentry.c` file of every test enables this selection.
2. The `<testname>testentry.c` file contains repeated calls to `val_execute_test` with different client-id and server-id endpoint ID.
3. `val_execute_test` invokes the client test function on the specified client endpoint which invokes the server test function on the specified server endpoint. This is how the test scenario is executed with the given client and server pair.
4. `val_execute_test` also checks the validity of the endpoint combination for the given target system.
5. It runs the test only when the client and server pair is valid for the target, otherwise it skips the check. For example, for an absent Non-secure hypervisor target, it would skip the VM to VM client-server pair combination.

2.5 Test endpoint boot

As part of initializing a test partition, the PM must program an entry into the first execution context of the test partition. This execution context is called the boot execution context.

For example, the hypervisor is responsible for initializing a VM. It initiates this process by programming an entry into the boot execution context corresponding to a vCPU of the VM. This vCPU is called primary boot CPU.

PM must follow this regardless of a partition has UP or MP execution contexts booting. For setting up the remaining execution contexts for the MP partition, see the [MP execution setup](#) section.

Using the primary boot CPU, test partition performs the following boot sequence:

1. Clears the Block Started by Symbol (BSS) region of the test partition image and fix the Global Offset Table (GOT) symbols of the image.
2. Programs the primary boot CPU stack to enable C programs.
3. Programs the VBAR with the default vector table.
4. Programs the GIC for handling partition interrupts.
5. Creates page table and enables the MMU
 - Flat mappings for endpoint regions are text, data, BSS, devices such as UART, NVM, and watchdog.
 - Support for 4K, 16K, 64K TT granule.
6. Gives control to the dispatcher function if the current endpoint is Dispatcher. Else, it calls `ffa_msg_wait` ABI to indicate that endpoints are initialized and ready to accept the request.



The EL0 SP would skip steps from 3 to 5.

2.6 MP execution context setup

PM initializes only the boot execution context. If the partition is an MP endpoint, initialization of other execution contexts (secondary vCPU) must be done through an **IMPLEMENTATION DEFINED** mechanism. For this, ACS relies on PAL APIs to initialize the other partition contexts.

The secondary execution context setup for Secure EP is as follows:

1. FF-A components in the Secure world do not perform power management independently from the Normal world. Instead, the SPMC, SPM Dispatcher (SPMD), and SPs are informed about Operating System Power Management (OSPM) operations initiated by the Normal world through Power State Coordination Interface (PSCI) functions.
2. The boot execution context of SP uses `FFA_SECONDARY_EP_REGISTER` interface to register the secondary execution context entry point with SPMC and SPMD for initialization during a secondary cold boot.
3. The secondary execution context of SP uses the registered entry point by SPMC programs when Normal world invokes `PSCI_CPU_ON` interface for the given vCPU.
4. Test SP then sets up the stack for the secondary vCPU. Note that test partition runs with MMU off on secondary vCPU.
5. SP uses `FFA_MSG_WAIT` interface to indicate completion of the secondary execution context to the FF-A framework.



For more information on MP execution context, see the [Arm Power State Coordination Interface](#) specification.

The execution context setup for ACS Non-secure EP is as follows:

1. The default implementation of PAL APIs relies on Power State Coordination Interface (PSCI) implementation of PM for the MP execution setups.
2. The primary boot vCPU of the partition uses the `PSCI_CPU_ON` interface to request the PM to initialize another vCPU of the VM or SP. These PAL APIs must be ported if you rely on a framework other than PSCI calls.
3. Upon invoking `PSCI_CPU_ON` interface, PM must release the mentioned secondary vCPU at the partition entry address specified during the `PSCI_CPU_ON` call.
4. Test partition sets up the stack for the secondary vCPU.



The test partition does not set up the MMU for the secondary vCPU, instead it performs the necessary cache maintenance operations for transactions to any shared locations between vCPUs.

5. Test partition uses the `PSCI_CPU_OFF` interface to exclude calling vCPU from the system for the given partition.

2.7 Error handling

This section defines the test methodology to handle error situations when they occur.

There are two types of error or fault conditions that are possible when running the tests:

1. In type 1, the generic code encounters an unexpected error situation from which before performing the authorized access.
2. In type 2, the test performs a sequence to trigger expected error conditions. For example, test endpoint performs unauthorized accesses in which it expects to trigger fault at the PM-level (abort must handle at EL2). The test does this to check the PM behavior for unauthorized accesses.

Test framework handles the type 1 errors as follows:

1. The framework relies on hardware watchdog and non-volatile memory region that must be assigned to Test SPI. In type 1, the generic code encounters an unexpected error situation from which it cannot recover or continue or spin.
2. The framework waits for watchdog timeout on encounter of error condition and expects watchdog to reset the system so that framework can continue with next available test.
3. The framework uses non-volatile memory to preserve test data over watchdog timer reset.

There are two ways to handle the type 2 errors:

1. If PM supports the injection of faults into originated endpoint (abort injection from higher EL to lower EL), error handling steps are as follows:
 - Test endpoint installs handler at the expected vector table location. For example, install synchronous abort handler for stage 2 Data Abort.
 - Test endpoint performs the authorized access and expects fault-handling at PM.
 - PM injects the fault at lower EL by copying ESR.EC and FAR system registers values.
 - Endpoint receives the abort at the installed handler which fixes the error condition and returns to interrupted code.
2. Use of type 1 error handling, hardware watchdog, and non-volatile memory region-based recovery.
 - Upon entry into test framework after rebooting, the framework reads the notification flag and checks whether the reboot was intended and marks the test status accordingly.

2.8 Running ACS with Linux OS

Following are the ways to run ACS tests along with Linux OS in the system.

- If hypervisor is present in the system and implements the primary scheduler, then ACS VM1 code can be launched as part of Linux OS kernel using kernel module programming. Also, VM2 (vm2.bin) and VM3 (vm3.bin) binaries can run as bare-metal VMs.
- If hypervisor is present in the system and Linux OS (EL1) implements the primary scheduler, then Linux OS runs as Primary Virtual Machine (PVM) and ACS VM1 (vm1.bin), VM2 (vm2.bin) and VM3 (vm3.bin) must run as secondary VMs and can also run as bare-metal VMs.
- If hypervisor is absent, then ACS VM1 code can be launched as part of Linux OS kernel using kernel module programming.



To run VM1 code as part of the Linux OS kernel, Linux kernel module files are required. These files are available in [linux-acs](#). To know the procedure to build and run FF-A tests on this configuration, see the [README](#).

2.9 Analyzing test results

Each test follows a uniform test structure that is defined by VAL.

- Performing any test initializations.
- Dispatching the test functions.
- Waiting for test completion.
- Performing the test exit.

The test can either pass, fail, skip, or be in an error state. For example, if a test times out or the system hangs, then it means that something went wrong and the test framework was unable to determine the error. In this case, you have to check the logs. If a test fails or skips, then you may see extra print messages to determine the cause.

The test suite summary is displayed at the end.

```
TEST: ffa_version SUITE: setup_discovery
Executing Test Setup client: VM1
ERROR: Check failed for iteration = 1
ERROR: Expected=0x10001 but Actual=0xffffffff
RESULT: FAILED (ERROR CODE=1)
```

```
TEST: ffa_id_get SUITE: setup_discovery
Executing Test Setup client: VM1
Executing Test Setup client: SP1
RESULT: PASSED
```

REGRESSION REPORT:

```
=====
TOTAL TESTS      : 2
TOTAL PASSED     : 1
TOTAL FAILED     : 1
TOTAL SKIPPED    : 0
TOTAL SIM ERROR  : 0
=====
***** END OF ACS *****
Entering standby..
```

Debugging a failing test

Since each test is organized with a logical set of self-checking code, if a failure occurs, searching for the relevant self-checking point is a useful point to start debugging.

Consider the above mentioned code snippet of a failing test on the display console.

Here are some points to consider when debugging.

- If the default prints do not give enough information, you can recompile and rerun the test binaries with high print verbosity level. See the test suite build README to understand how the test verbosity can be changed.
- Prints from each of the test endpoints are prefixed with the endpoint name. For example, print from SP1 is prefixed with "SP1: "
- In case of a test fail,
 - Along with the error message, the test also prints the file and line number from where the error message is printed.
 - Test results contain the error code associated with the error message. The status of the error code is mapped with a structure `val_status_t` that is available at `val/inc/val.h`. Look for the `enum` that is dedicated to this number to see the status in the verbatim form.

Proprietary Notice

This document is protected by copyright and other related rights and the use or implementation of the information contained in this document may be protected by one or more patents or pending patent applications. No part of this document may be reproduced in any form by any means without the express prior written permission of Arm Limited ("Arm"). No license, express or implied, by estoppel or otherwise to any intellectual property rights is granted by this document unless specifically stated.

Your access to the information in this document is conditional upon your acceptance that you will not use or permit others to use the information for the purposes of determining whether the subject matter of this document infringes any third party patents.

The content of this document is informational only. Any solutions presented herein are subject to changing conditions, information, scope, and data. This document was produced using reasonable efforts based on information available as of the date of issue of this document. The scope of information in this document may exceed that which Arm is required to provide, and such additional information is merely intended to further assist the recipient and does not represent Arm's view of the scope of its obligations. You acknowledge and agree that you possess the necessary expertise in system security and functional safety and that you shall be solely responsible for compliance with all legal, regulatory, safety and security related requirements concerning your products, notwithstanding any information or support that may be provided by Arm herein. In addition, you are responsible for any applications which are used in conjunction with any Arm technology described in this document, and to minimize risks, adequate design and operating safeguards should be provided for by you.

This document may include technical inaccuracies or typographical errors. THIS DOCUMENT IS PROVIDED "AS IS". ARM PROVIDES NO REPRESENTATIONS AND NO WARRANTIES, EXPRESS, IMPLIED OR STATUTORY, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTIES OF MERCHANTABILITY, SATISFACTORY QUALITY, NON-INFRINGEMENT OR FITNESS FOR A PARTICULAR PURPOSE WITH RESPECT TO THE DOCUMENT. For the avoidance of doubt, Arm makes no representation with respect to, and has undertaken no analysis to identify or understand the scope and content of, any patents, copyrights, trade secrets, trademarks, or other rights.

TO THE EXTENT NOT PROHIBITED BY LAW, IN NO EVENT WILL ARM BE LIABLE FOR ANY DAMAGES, INCLUDING WITHOUT LIMITATION ANY DIRECT, INDIRECT, SPECIAL, INCIDENTAL, PUNITIVE, OR CONSEQUENTIAL DAMAGES, HOWEVER CAUSED AND REGARDLESS OF THE THEORY OF LIABILITY, ARISING OUT OF ANY USE OF THIS DOCUMENT, EVEN IF ARM HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Reference by Arm to any third party's products or services within this document is not an express or implied approval or endorsement of the use thereof.

This document consists solely of commercial items. You shall be responsible for ensuring that any permitted use, duplication, or disclosure of this document complies fully with any relevant export laws and regulations to assure that this document or any portion thereof is not exported, directly or indirectly, in violation of such export laws. Use of the word "partner" in reference to Arm's customers is not intended to create or refer to any partnership relationship with any other company. Arm may make changes to this document at any time and without notice.

This document may be translated into other languages for convenience, and you agree that if there is any conflict between the English version of this document and any translation, the terms of the English version of this document shall prevail.

The validity, construction and performance of this notice shall be governed by English Law.

The Arm corporate logo and words marked with ® or ™ are registered trademarks or trademarks of Arm Limited (or its affiliates) in the US and/or elsewhere. Please follow Arm's trademark usage guidelines at <https://www.arm.com/company/policies/trademarks>. All rights reserved. Other brands and names mentioned in this document may be the trademarks of their respective owners.

Arm Limited. Company 02557590 registered in England.

110 Fulbourn Road, Cambridge, England CB1 9NJ.

PRE-1121-V1.0

Product and document information

Read the information in these sections to understand the release status of the product and documentation, and the conventions used in the Arm documents.

Product status

All products and Services provided by Arm require deliverables to be prepared and made available at different levels of completeness. The information in this document indicates the appropriate level of completeness for the associated deliverables.

Product completeness status

The information in this document is Final, that is for a developed product.

Revision history

These sections can help you understand how the document has changed over time.

Document release information

The Document history table gives the issue number and the released date for each released issue of this document.

Document history

Issue	Date	Confidentiality	Change
0102-02	20 July 2026	Non-Confidential	v1.2 EAC release
0102-01	26 November 2024	Non-Confidential	v1.2 EAC release
0100-01	6 August 2024	Non-Confidential	v1.1 EAC release
0009-00	26 June 2024	Non-Confidential	v1.1 Beta release
0008-04	5 March 2024	Non-Confidential	v1.1 Beta1 release
0008-03	2 August 2022	Non-Confidential	v1.1 Beta0 release
0008-02	30 July 2021	Non-Confidential	v1.0 Beta release
0005-01	6 April 2021	Non-Confidential	Alpha release

The Change history tables describe the technical changes between released issues of this document in reverse order. Issue numbers match the revision history in [Document release information](#) on page 22.

Table 2: Differences between Issue 0102-01 and Issue 0102-02

Change	Location
Added a new abbreviation TPM CRB.	1.1 Abbreviations on page 4
Updated the test suite categories and their descriptions table.	1.3 Architecture Compliance Suite on page 5
Updated the directory structure figure and the test suit directory table.	1.5 Directory structure on page 6

Table 3: Differences between Issue 0100-01 and Issue 0102-01

Change	Location
Removed the test suite organization section.	-

Table 4: Differences between Issue 0009-00 and Issue 0100-01

Change	Location
Updated the test suite summary.	See 2.9 Analyzing test results on page 18.

Table 5: Differences between Issue 0008-04 and Issue 0009-00

Change	Location
Added figures SP4 to SUT when Non-secure hypervisor is present and SUT when Non-secure hypervisor is absent.	See 2.1 Test layering details on page 9.

Table 6: Differences between Issue 0008-03 and Issue 0008-04

Change	Location
Added two new directories test_database_v_1_0.c and test_database_v_1_1.c in Arm FF-A test suite directory structure figure and table.	See Test suite organization .
Updated the component name from test.CMake to test.cmake in Arm FF-A test suite directory details table.	See Test suite organization .

Table 7: Differences between Issue 0008-02 and Issue 0008-03

Change	Location
Added two test suites - Notifications and Interrupts.	See 1.3 Architecture Compliance Suite on page 5.
The Arm FF-A test suite directory structure figure is updated. v1.0 and v1.1 rows are added to the Arm FF-A test suite directory details table.	See Test suite organization .

Table 8: Differences between Issue 0005-01 and Issue 0008-02

Change	Location
Added the terms PVM and SVM in the abbreviations table.	See 1.1 Abbreviations on page 4.
Updated the documentation section for test suite components.	See 1.4 Test suite components on page 6.
Added the linux_app file and its description in the directory structure.	See 1.5 Directory structure on page 6.
Updated the integration steps for Normal world Test dispatcher endpoint (VM1) binary.	See 2.2 Integrating the test suite with the SUT on page 10.
Updated the Running ACS with Linux OS section.	See 2.8 Running ACS with Linux OS on page 17.

Table 9: Issue 0005-01

Change	Location
First release.	-

Conventions

The following subsections describe conventions used in Arm documents.

Glossary

The Arm Glossary is a list of terms used in Arm documentation, together with definitions for those terms. The Arm Glossary does not contain terms that are industry standard unless the Arm meaning differs from the generally accepted meaning.

See the Arm Glossary for more information: developer.arm.com/glossary.

Typographic conventions

Arm documentation uses typographical conventions to convey specific meaning.

Convention	Use
italic	Citations.
bold	Interface elements, such as menu names. Terms in descriptive lists, where appropriate.
monospace	Text that you can enter at the keyboard, such as commands, file and program names, and source code.
monospace <u>underline</u>	A permitted abbreviation for a command or option. You can enter the underlined text instead of the full command or option name.
<and>	Encloses replaceable terms for assembler syntax where they appear in code or code fragments. For example: MRC p15, 0, <Rd>, <CRn>, <CRm>, <Opcode_2>
SMALL CAPITALS	Terms that have specific technical meanings as defined in the <i>Arm® Glossary</i> . For example, IMPLEMENTATION DEFINED , IMPLEMENTATION SPECIFIC , UNKNOWN , and UNPREDICTABLE .



We recommend the following. If you do not follow these recommendations your system might not work.



Your system requires the following. If you do not follow these requirements your system will not work.



You are at risk of causing permanent damage to your system or your equipment, or of harming yourself.



This information is important and needs your attention.



This information might help you perform a task in an easier, better, or faster way.



This information reminds you of something important relating to the current content.

Useful resources

This document contains information that is specific to this product. See the following resources for other useful information.

Arm documents are available on developer.arm.com/documentation.

Confidential documents are only available to licensees, when logged in. Each document link in the following tables provides direct access to the online version of the document.

Table 1: Arm publications

Document name	Document ID	Confidentiality
Arm® Firmware Framework for Arm A-profile	DEN0077A	Non-Confidential
Arm® System Memory Management Unit Architecture Specification SMMU – architecture versions 2.0	ARM IHI 0062D.c (ID070116)	Non-Confidential
Arm® SMC Calling Convention	DEN 0028D	on-Confidential
Arm® Architecture Reference Manual for A-profile architecture	DDI 0487	Non-Confidential
Arm® Power State Coordination Interface Platform Design Document	DEN0022D	Non-Confidential
TPM Service Command Response Buffer Interface Over FF-A	DEN0138	Non-confidential